

A Technical Introduction to Proximal policy optimization

Section 1

Proximal policy optimization (PPO) is a reinforcement learning (RL) algorithm for training an intelligent agent. Specifically, it is a policy gradient method, often used for deep RL when the policy network is very large.

Section 2

for $k = 0, 1, 2, \dots$ do Collect set of trajectories $D_k = \{ \tau_i \}$ by running policy $\pi_k = \pi(\theta_k)$ in the environment. Compute rewards-to-go $\hat{R}_t$. Compute advantage estimates, A_t (using any method of advantage estimation) based on the current value function V_{ϕ_k} . Estimate policy gradient as $g_k = \frac{1}{|D_k|} \sum_{\tau \in D_k} \sum_{t=0}^{T-1} \nabla_{\theta} \log \pi(\theta(s_t)) \cdot A_t$.

Section 3

PPO objective function The objective function of PPO takes the expectation operator (denoted as E) which means that this function will be computed over quantities of trajectories. The expectation operator takes the minimum of two terms:

Section 4

Policy gradient laws: the advantage function The advantage function (denoted as A) is central to PPO, as it tries to answer the question of whether a specific action of the agent is better or worse than some other possible action in a given state. By definition, the advantage function is an estimate of the relative value for a selected action. If the output of this function is positive, it means that the action in question is better than the average return, so the possibilities of selecting that specific action will increase. The opposite is true for a negative advantage output. The advantage function can be defined as $A = Q - V$, where Q is the discounted sum of rewards (the total weighted reward for the completion of an episode) and V is the baseline estimate. Since the advantage function is calculated after the completion of an episode, the program records the outcome of the episode. Therefore, calculating advantage is essentially an unsupervised learning problem. The baseline estimate comes from the value function that outputs the expected discounted sum of an episode starting from the current state. In the PPO algorithm, the

baseline estimate will be noisy (with some variance), as it also uses a neural network, like the policy function itself. With Q and V computed, the advantage function is calculated by subtracting the baseline estimate from the actual discounted return. If $A > 0$, the actual return of the action is better than the expected return from experience; if $A < 0$, the actual return is worse.

Section 5

History The predecessor to PPO, Trust Region Policy Optimization (TRPO), was published in 2015. It addressed the instability issue of another algorithm, the Deep Q-Network (DQN), by using the trust region method to limit the KL divergence between the old and new policies. However, TRPO uses the Hessian matrix (a matrix of second derivatives) to enforce the trust region, but the Hessian is inefficient for large-scale problems. PPO was published in 2017. It was essentially an approximation of TRPO that does not require computing the Hessian. The KL divergence constraint was approximated by simply clipping the policy gradient. Since 2018, PPO was the default RL algorithm at OpenAI. PPO has been applied to many areas, such as controlling a robotic arm, beating professional players at Dota 2 (OpenAI Five), and playing Atari games.

Section 6

$\text{clip} \left(\frac{r_t(\theta)}{r_t(\theta_{\text{old}})} \cdot A \right)$: the policy ratio is first clipped to the range $[1 - \epsilon, 1 + \epsilon]$; generally, ϵ is defined to be 0.2. Then, as before, it is multiplied by the advantage. The fundamental intuition behind PPO is the same as that of TRPO: conservatism. Clipping results in a conservative advantage estimate of the new policy. The reasoning is that if an agent makes significant changes due to high advantage estimates, its policy update will be large and unstable, and may diverge from the optimal policy with little possibility of recovery. There are two common applications of the clipping function: when an action under a new policy happens to be a good choice based on the advantage function, the clipping function limits how much credit can be given to a new policy for up-weighted good actions. On the other hand, when an action under the old policy is judged to be bad, the clipping function constrains how much the agent can accept the down-weighted bad actions of the new policy. Consequently, the clipping mechanism is designed to discourage the incentive of moving beyond the defined range by clipping both directions. The advantage of this method is that it can be optimized directly with gradient descent, as opposed to the strict KL divergence constraint of TRPO, making the implementation faster and more intuitive. After computing the clipped surrogate objective function, the agent has two probability ratios: one non-clipped and one clipped. Then, by taking the minimum

of the two objectives, the final objective becomes a lower bound (a pessimistic bound) of what the agent knows is possible. In other words, the minimum method makes sure that the agent is doing the safest possible update.

Section 7

Simplicity PPO approximates what TRPO does, with considerably less computation. It uses first-order optimization (the clip function) to constrain the policy update, while TRPO uses KL divergence constraints (second-order optimization). Compared to TRPO, the PPO method is relatively easy to implement and requires less computational resource and time. Therefore, it is cheaper and more efficient to use PPO in large-scale problems.